

Library Management System - Complete Documentation

Table of Contents

1. [System Overview](#)
 2. [Features & Capabilities](#)
 3. [System Architecture](#)
 4. [Database Schema](#)
 5. [Project Structure](#)
 6. [Installation Guide](#)
 7. [API Documentation](#)
 8. [Testing with Postman](#)
 9. [User Workflows](#)
 10. [Troubleshooting](#)
-

System Overview

The **Library Management System** is a comprehensive web-based application designed to manage library operations including book inventory, patron management, borrowing transactions, reservations, fines, and multi-role user access control.

What the System Can Do

Core Functionalities:

- Multi-role authentication (Admin, Faculty, Client/Student)
- Complete book inventory management with advanced search
- Patron registration and profile management
- Borrowing workflow with pending approval system
- Book reservation system for unavailable books
- Automated fine calculation for overdue books
- Activity logging and login history tracking
- Real-time analytics and statistics

- Session management and security features
- Faculty-specific permissions and extended borrowing privileges

Key Highlights:

- **Pending Approval Workflow:** Faculty can request books, admins approve/reject
 - **Smart Reservations:** Automatic notification when reserved books become available
 - **Fine Management:** Automated calculation with waiver capabilities
 - **Comprehensive Logging:** Track all user activities and system events
 - **Security Features:** Account lockout, session timeout, two-factor authentication support
 - **Flexible Settings:** Configurable borrowing periods, fine rates, and limits
-

Features & Capabilities

For Administrators

User Management

- Create, edit, and delete user accounts
- Assign roles (admin, faculty, client)
- Link users to patron records
- Manage faculty permissions
- View user activity logs and login history

Book Management

- Add, edit, and delete books
- Track quantity and availability
- Manage book categories and tags
- Full-text search capabilities
- Import/export book records

Borrowing Management

- Approve/reject faculty borrow requests

- Process book checkouts and returns
- Mark books as overdue or lost
- View borrowing history and statistics
- Generate reports

Reservation Management

- View all reservations
- Process reservation fulfillment
- Cancel or expire reservations
- Notify patrons when books are ready

Fine Management

- View all fines (paid, unpaid, partial)
- Process fine payments
- Waive fines with reason tracking
- Generate fine reports
- Set fine calculation rules

Analytics & Reports

- Dashboard with key metrics
- Borrowing trends and statistics
- Overdue book reports
- Popular books analysis
- User activity summaries

For Faculty Members

Book Search & Discovery

- Advanced search with filters
- Browse by category, author, or tags
- View book availability status

- Check book details and location

Borrowing

- Request to borrow books (pending approval)
- View borrowing history
- Check due dates and extend loans
- View current borrowed books
- Extended borrowing periods (up to 30 days)
- Higher borrowing limits (up to 10 books)

Reservations

- Reserve unavailable books
- View reservation status
- Get notifications when books are ready
- Cancel reservations

Profile Management

- Update personal information
- View borrowing statistics
- Check fine balance

For Clients/Students

Book Search

- Search available books
- View book details
- Check availability

Borrowing

- Borrow available books (instant checkout)
- View current borrowed books
- Check due dates

- View borrowing history
- Standard borrowing period (14 days)
- Maximum 5 books at a time

Account Management

- View profile information
 - Check fine balance
 - View borrowing statistics
-

System Architecture

Technology Stack

Backend:

- PHP 7.4+
- MySQL 8.0+
- Character Set: UTF8MB4 (full Unicode support)

Frontend:

- HTML5
- CSS3
- JavaScript (Vanilla)
- Bootstrap (for UI components)

APIs:

- RESTful API architecture
- JSON response format
- Token-based authentication

Security Features

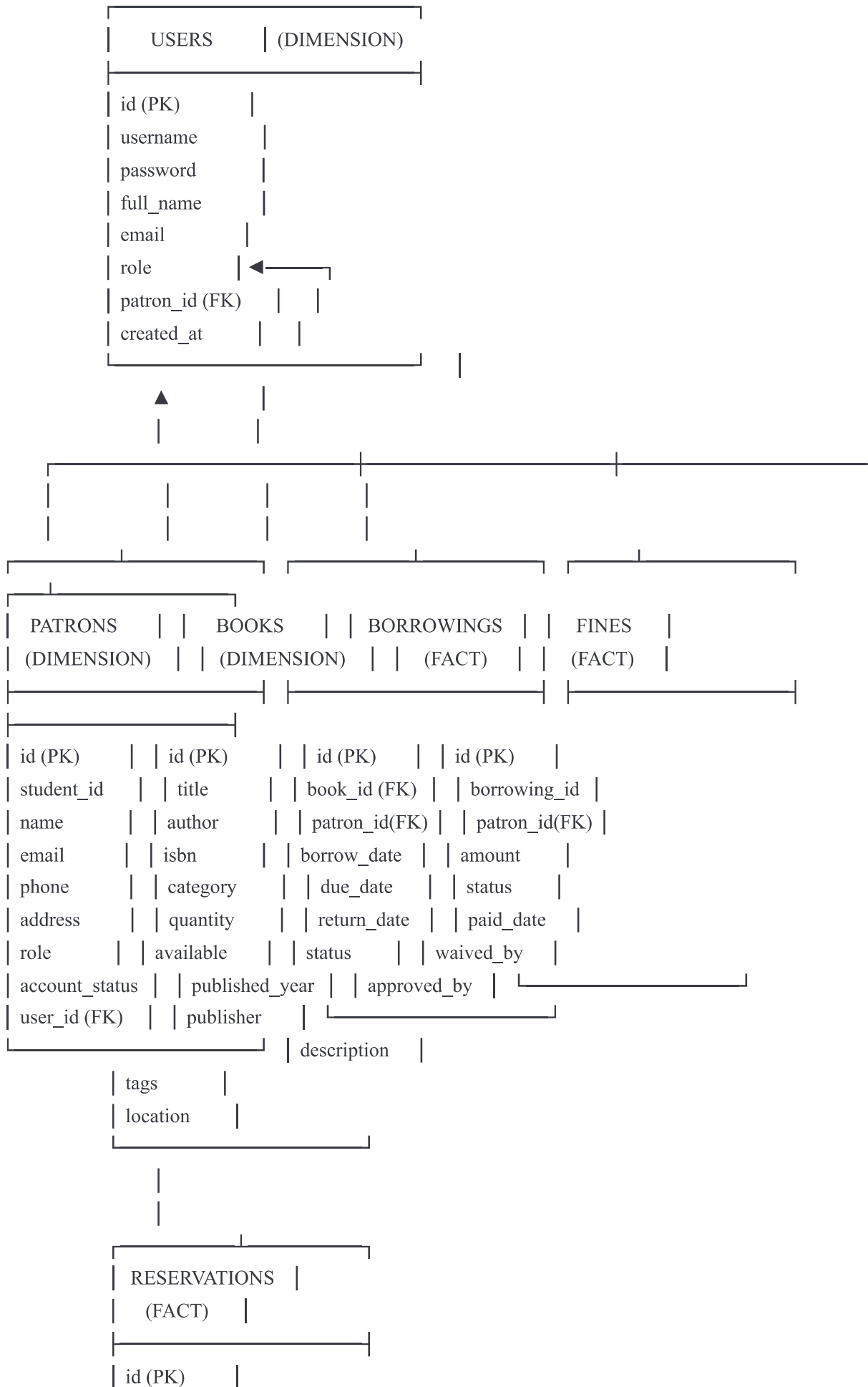
- Password hashing (bcrypt)

- SQL injection prevention (prepared statements)
 - XSS protection
 - CSRF protection
 - Session management
 - Account lockout after failed attempts
 - Two-factor authentication support
 - IP tracking and user agent logging
-

Database Schema

Star Schema Design

The system uses an **enhanced star schema** with core dimension tables and multiple fact tables for different operations.



	book_id (FK)	
	user_id (FK)	
	patron_id (FK)	
	reservation_date	
	expiry_date	
	status	
	notified	

Core Tables

1. USERS (Dimension Table)

Purpose: Central user authentication and authorization

Column	Type	Description
id	INT (PK)	Unique user identifier
username	VARCHAR(50)	Login username (unique)
password	VARCHAR(255)	Hashed password
full_name	VARCHAR(100)	User's full name
email	VARCHAR(100)	Email address (unique)
role	ENUM	admin, client, faculty
patron_id	INT (FK)	Links to patrons table
reset_token	VARCHAR(64)	Password reset token
reset_expiry	DATETIME	Token expiration
two_factor_secret	VARCHAR(32)	2FA secret key
two_factor_enabled	TINYINT(1)	2FA status
last_login	DATETIME	Last successful login
last_activity	DATETIME	Last activity timestamp
login_attempts	INT	Failed login counter
locked_until	DATETIME	Account lockout timestamp
created_at	TIMESTAMP	Account creation date

Indexes: role, username, email, patron_id

2. PATRONS (Dimension Table)

Purpose: Store patron/member information

Column	Type	Description
id	INT (PK)	Unique patron identifier
student_id	VARCHAR(50)	Student/Member ID (unique)
name	VARCHAR(100)	Patron's full name
email	VARCHAR(100)	Contact email (unique)
phone	VARCHAR(20)	Contact phone number
address	TEXT	Residential address
role	ENUM	client, faculty
account_status	ENUM	pending, registered, active, suspended
user_id	INT (FK)	Links to users table
created_at	TIMESTAMP	Registration date
updated_at	TIMESTAMP	Last update timestamp

Indexes: email, student_id, account_status, user_id

3. BOOKS (Dimension Table)

Purpose: Complete book inventory

Column	Type	Description
id	INT (PK)	Unique book identifier
title	VARCHAR(200)	Book title
author	VARCHAR(100)	Book author(s)
isbn	VARCHAR(20)	ISBN number (unique)
category	VARCHAR(50)	Book category/genre
quantity	INT	Total copies owned
available	INT	Currently available copies
published_year	INT	Year of publication
publisher	VARCHAR(100)	Publisher name
description	TEXT	Book description
tags	VARCHAR(500)	Searchable tags
location	VARCHAR(100)	Physical location in library
created_at	TIMESTAMP	Date added to system
updated_at	TIMESTAMP	Last update timestamp

Indexes: title, author, category, isbn, available **Full-text Index:** title, author, isbn, tags

4. BORROWINGS (Fact Table)

Purpose: Track all borrowing transactions

Column	Type	Description
id	INT (PK)	Unique borrowing record ID
book_id	INT (FK)	References books table
patron_id	INT (FK)	References patrons table
borrow_date	DATE	Date borrowed
due_date	DATE	Due date for return
return_date	DATE	Actual return date
status	ENUM	pending, borrowed, returned, overdue, lost, rejected
notes	TEXT	Additional notes
approved_by	INT (FK)	Admin who approved (for faculty)
created_at	TIMESTAMP	Request creation date
updated_at	TIMESTAMP	Last update timestamp

Status Values:

- `pending`: Faculty request awaiting approval
- `borrowed`: Currently borrowed
- `returned`: Book returned
- `overdue`: Past due date
- `lost`: Book reported lost
- `rejected`: Request denied by admin

Indexes: status, book_id, patron_id, borrow_date, due_date, return_date

5. BOOK_RESERVATIONS (Fact Table)

Purpose: Manage book reservations

Column	Type	Description
id	INT (PK)	Unique reservation ID
book_id	INT (FK)	References books table
user_id	INT (FK)	References users table
patron_id	INT (FK)	References patrons table
reservation_date	DATETIME	When reserved
expiry_date	DATETIME	Reservation expiration
status	ENUM	pending, ready, fulfilled, cancelled, expired
notes	TEXT	Additional notes
notified	TINYINT(1)	Notification sent flag
notification_date	DATETIME	When notified
created_at	TIMESTAMP	Creation timestamp
updated_at	TIMESTAMP	Last update timestamp

Indexes: status, user_id, book_id, expiry_date

6. FINES (Fact Table)

Purpose: Track overdue fines and payments

Column	Type	Description
id	INT (PK)	Unique fine ID
borrowing_id	INT (FK)	References borrowings table
patron_id	INT (FK)	References patrons table
amount	DECIMAL(10,2)	Fine amount
reason	VARCHAR(255)	Reason for fine
days_overdue	INT	Number of days overdue
status	ENUM	unpaid, paid, waived, partial
amount_paid	DECIMAL(10,2)	Amount paid so far
paid_date	DATETIME	Payment date
waived_by	INT (FK)	Admin who waived fine
waived_reason	TEXT	Reason for waiving
created_at	TIMESTAMP	Fine creation date
updated_at	TIMESTAMP	Last update timestamp

Indexes: status, patron_id, borrowing_id

--

Supporting Tables

7. FACULTY_PERMISSIONS

Purpose: Store faculty-specific permissions and limits

Column	Type	Description
id	INT (PK)	Permission record ID
user_id	INT (FK)	References users table
can_search_books	TINYINT(1)	Search permission
can_borrow_books	TINYINT(1)	Borrowing permission
can_view_own_history	TINYINT(1)	View history permission
can_make_reservations	TINYINT(1)	Reservation permission
max_borrow_limit	INT	Max books allowed (default: 10)
borrow_days_override	INT	Custom borrowing period

8. ACTIVITY_LOGS

Purpose: Track all user actions in the system

Column	Type	Description
id	INT (PK)	Log entry ID
user_id	INT (FK)	References users table
action	VARCHAR(100)	Action performed
description	TEXT	Detailed description
ip_address	VARCHAR(45)	User's IP address
user_agent	TEXT	Browser/device info
created_at	TIMESTAMP	Action timestamp

9. LOGIN_HISTORY

Purpose: Track login attempts (successful and failed)

Column	Type	Description
id	INT (PK)	History entry ID
user_id	INT (FK)	References users table
username	VARCHAR(50)	Username used
ip_address	VARCHAR(45)	Login IP address
user_agent	TEXT	Browser/device info
status	ENUM	success, failed, blocked
failure_reason	VARCHAR(255)	Reason for failure
created_at	TIMESTAMP	Login attempt time

10. USER_SESSIONS

Purpose: Manage active user sessions

Column	Type	Description
id	INT (PK)	Session ID
user_id	INT (FK)	References users table
session_id	VARCHAR(128)	Unique session identifier
ip_address	VARCHAR(45)	Session IP
user_agent	TEXT	Browser info
last_activity	DATETIME	Last activity timestamp
created_at	TIMESTAMP	Session start time

11. USER_PROFILES

Purpose: Extended user profile information

Column	Type	Description
id	INT (PK)	Profile ID
user_id	INT (FK)	References users table
phone	VARCHAR(20)	Phone number
address	TEXT	Address
date_of_birth	DATE	Birth date
profile_picture	VARCHAR(255)	Photo path
bio	TEXT	Biography
department	VARCHAR(100)	Department/Faculty
membership_status	ENUM	active, suspended, inactive

12. USER_BORROWING_STATS

Purpose: Aggregate borrowing statistics per user

Column	Type	Description
id	INT (PK)	Stats record ID
user_id	INT (FK)	References users table
total_borrowed	INT	Total books borrowed
currently_borrowed	INT	Current borrowed count
total_returned	INT	Total books returned
overdue_count	INT	Current overdue count
total_fines	DECIMAL(10,2)	Total fines amount
last_borrow_date	DATETIME	Last borrow date
updated_at	TIMESTAMP	Last update

13. SYSTEM_SETTINGS

Purpose: Configurable system parameters

Column	Type	Description
id	INT (PK)	Setting ID
setting_key	VARCHAR(100)	Unique setting key
setting_value	VARCHAR(255)	Setting value
setting_type	ENUM	string, number, boolean, json
description	TEXT	Setting description
category	VARCHAR(50)	Setting category
is_editable	TINYINT(1)	Can be edited

Default Settings:

- `default_borrow_days`: 14
 - `faculty_max_borrow_days`: 30
 - `max_books_per_patron`: 5
 - `faculty_max_books`: 10
 - `reservation_expiry_days`: 3
 - `fine_per_day`: 5.00
 - `max_fine_amount`: 500.00
 - `grace_period_days`: 0
 - `max_login_attempts`: 5
 - `account_lockout_duration`: 30
 - `session_timeout`: 30
-

Database Views

view_current_borrowings

Shows all active borrowings with complete details including patron and book information.

view_pending_faculty_requests

Lists all pending borrow requests from faculty members awaiting admin approval.

view_overdue_books

Displays all overdue books with days overdue and associated fine information.

view_book_availability

Summary of book availability status across the entire inventory.

Database Triggers

Update Book Availability

- **On Borrow Approval:** Decreases available count when status changes from pending to borrowed
- **On Return:** Increases available count when book is returned
- **On Pending Request:** Does not affect availability (only affects when approved)

Update Borrowing Statistics

- **On Borrow:** Increments total_borrowed and currently_borrowed
- **On Return:** Decrements currently_borrowed, increments total_returned
- **On Overdue:** Increments overdue_count

Reservation Management

- **On Book Return:** Automatically updates first pending reservation to "ready" status

Faculty Permissions

- **On User Creation:** Automatically creates faculty_permissions record when role is "faculty"
-

Stored Procedures

checkPatronCanBorrow

Comprehensive check before allowing a patron to borrow:

- Verifies borrowing limit not exceeded
- Checks for overdue books
- Validates unpaid fines
- Returns boolean can_borrow status

getFacultyPermissions

Retrieves faculty member's permissions and limits.

Project Structure

```
Library/
|
|— admin/                # Admin panel files
| |— active_sessions.php  # View active user sessions
| |— activity_logs.php    # System activity logs
| |— add_book.php         # Add new books
| |— add_patron.php       # Add new patrons
| |— analytics.php        # System analytics dashboard
| |— api_book_search.php  # Book search API endpoint
| |— books.php            # Book management
| |— borrowing.php        # Borrowing management
| |— check_patron_limit.php # Check borrowing limits
| |— dashboard.php        # Admin dashboard
| |— edit_book.php        # Edit book details
| |— edit_patron.php      # Edit patron details
| |— fines.php            # Fine management
| |— fix_availability.php  # Fix availability issues
| |— import_patrons.php   # Bulk patron import
| |— inventory.php        # Inventory management
| |— login_history.php    # User login history
| |— manage_faculty.php   # Faculty management
| |— new_arrivals.php     # New arrivals management
| |— patron_directory.php # Patron directory
```

- | | — patron_profile.php # Individual patron profile
- | | — print_records.php # Print/export records
- | | — search_books.php # Advanced book search
- | | — settings.php # System settings
- | | — sync_patron_user_data.php # Sync patron-user data
- | | — test_api.php # API testing interface
- |
- | — auth/ # Authentication files
- | | — check_auth.php # Authentication check
- | | — forgot_password.php # Password recovery
- | | — login.php # Login page
- | | — logout.php # Logout handler
- | | — register.php # User registration
- | | — reset_password.php # Password reset
- |
- | — client/ # Client/Student portal
- | | — chatbot.php # AI chatbot assistant
- | | — dashboard.php # Client dashboard
- | | — search_books.php # Book search interface
- |
- | — config/ # Configuration files
- | | — config.php # Database configuration
- |
- | — documentation/ # System documentation
- | | — [Documentation files]
- |
- | — faculty/ # Faculty portal
- | | — borrow_request.php # Submit borrow request
- | | — confirm_reservation.php # Confirm reservation
- | | — dashboard.php # Faculty dashboard
- | | — my_borrowings.php # View borrowing history
- | | — profile.php # Faculty profile
- | | — reservations.php # View reservations
- | | — reserve_book.php # Reserve a book
- | | — search_books.php # Search books
- |
- | — includes/ # Reusable PHP includes
- | | — api_books.php # Books API functions
- | | — book_api.php # Book management API
- | | — chatbot_api.php # Chatbot API
- | | — chatbot_api_client.php # Client chatbot API
- | | — functions.php # Helper functions
- | | — sidebar.php # Sidebar component
- | | — SimpleXLSX.php # Excel file parser

```
|
|
|— test_api/           # API testing files
|   └── [Test files]
|
|
|— database_setup.sql   # Database schema SQL
|— home.php            # Landing page
|— index.php           # Main entry point
|— admin_test_api.html  # Admin API tester
|— chatbot.php          # Chatbot interface
|— simple_test_api.html # Simple API tester
|— test_api.html        # Full API tester
```

Installation Guide

Prerequisites

- **XAMPP/WAMP/MAMP** (Apache + MySQL + PHP 7.4+)
 - **MySQL 8.0+** or **MariaDB 10.5+**
 - **Web Browser** (Chrome, Firefox, Edge)
 - **Postman** (for API testing)
 - **Text Editor** (VS Code, Sublime Text, etc.)
-

Step 1: Install XAMPP

1. Download XAMPP from <https://www.apachefriends.org>
 2. Install XAMPP to `C:\xampp` (Windows) or `/Applications/XAMPP` (Mac)
 3. Start Apache and MySQL services from XAMPP Control Panel
-

Step 2: Create Database

1. Open web browser and navigate to `http://localhost/phpmyadmin`
2. Click "New" to create a new database
3. Database name: `library_system`

4. Collation: `utf8mb4_unicode_ci`

5. Click "Create"

Step 3: Import Database Schema

Option A: Using phpMyAdmin

1. Select `library_system` database
2. Click "Import" tab
3. Choose file: `database_setup.sql`
4. Click "Go" to execute

Option B: Using MySQL Command Line

```
bash

mysql -u root -p library_system < database_setup.sql
```

Verify Installation:

```
sql

USE library_system;
SHOW TABLES;
```

You should see 13 tables created.

Step 4: Configure Application

1. Navigate to project folder
2. Open `config/config.php`
3. Update database credentials:

```
php
```

```
<?php
// Database Configuration
define('DB_HOST', 'localhost');
define('DB_USER', 'root');
define('DB_PASS', ''); // Your MySQL password
define('DB_NAME', 'library_system');
define('DB_CHARSET', 'utf8mb4');

// Create connection
$conn = new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME);

// Set charset to utf8mb4
$conn->set_charset(DB_CHARSET);

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
?>
```

Step 5: Deploy Application

1. Copy the entire `Library` folder to `C:\xampp\htdocs\`
 2. Full path should be: `C:\xampp\htdocs\Library\`
 3. Verify folder structure matches the project structure above
-

Step 6: Create Default Admin Account

Run this SQL in phpMyAdmin:

```
sql
```

-- Create admin user

```
INSERT INTO users (username, password, full_name, email, role, created_at)
VALUES (
  'admin',
  '$2y$10$92IXUNpkjO0rQQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi', -- password: 'password'
  'System Administrator',
  'admin@library.com',
  'admin',
  NOW()
);
```

-- Create sample patron

```
INSERT INTO patrons (student_id, name, email, phone, role, account_status)
VALUES (
  'ADM-001',
  'System Administrator',
  'admin@library.com',
  '09171234567',
  'client',
  'active'
);
```

-- Link user to patron

```
UPDATE users
SET patron_id = (SELECT id FROM patrons WHERE email = 'admin@library.com' LIMIT 1)
WHERE username = 'admin';
```

Step 7: Test Installation

1. Open browser: `http://localhost/Library/`
 2. You should see the login page
 3. Login with:
 - **Username:** `admin`
 - **Password:** `password`
 4. If successful, you'll see the admin dashboard
-

Step 8: Change Default Password

IMPORTANT: Change the default admin password immediately!

1. Go to Settings or Profile
 2. Click "Change Password"
 3. Enter current password: `password`
 4. Enter new secure password
 5. Save changes
-

Troubleshooting Installation

Problem: "Connection failed" error

- Solution: Check MySQL is running in XAMPP
- Verify database credentials in `config/config.php`

Problem: "Table doesn't exist" error

- Solution: Re-run `database_setup.sql`
- Verify database name is `library_system`

Problem: "Access denied" error

- Solution: Check MySQL username/password
- Default XAMPP: username=`root`, password=`empty`

Problem: Character encoding issues

- Solution: Ensure database collation is `utf8mb4_unicode_ci`
 - Check `config.php` has `$conn->set_charset('utf8mb4');`
-

API Documentation

Base URL

http://localhost/Library/

Authentication

Most API endpoints require authentication via session cookies. Login first at </auth/login.php>.

Authentication Endpoints

1. User Registration

http

POST /auth/register.php

Request Body (JSON):

```
json
{
  "username": "johndoe",
  "password": "SecurePass123!",
  "full_name": "John Doe",
  "email": "john.doe@email.com",
  "role": "client"
}
```

Response (Success):

```
json
{
  "success": true,
  "message": "Registration successful",
  "user_id": 15
}
```

2. User Login

http

POST /auth/login.php

Request Body (JSON):

```
json

{
  "username": "admin",
  "password": "password"
}
```

Response (Success):

```
json

{
  "success": true,
  "message": "Login successful",
  "user": {
    "id": 1,
    "username": "admin",
    "full_name": "System Administrator",
    "role": "admin",
    "email": "admin@library.com"
  },
  "redirect": "/admin/dashboard.php"
}
```

Response (Failed):

```
json

{
  "success": false,
  "message": "Invalid username or password",
  "attempts_remaining": 3
}
```

3. Logout

```
http  
GET /auth/logout.php
```

Response:

```
json  
  
{  
  "success": true,  
  "message": "Logged out successfully"  
}
```

4. Forgot Password

```
http  
POST /auth/forgot_password.php
```

Request Body:

```
json  
  
{  
  "email": "john.doe@email.com"  
}
```

5. Reset Password

```
http  
POST /auth/reset_password.php
```

Request Body:

```
json
```

```
{  
  "token": "abc123token456",  
  "password": "NewSecurePass123!",  
  "confirm_password": "NewSecurePass123!"  
}
```

Book Management Endpoints

1. Get All Books

http

GET /includes/api_books.php?action=list

Response:

```
json  
  
{  
  "success": true,  
  "books": [  
    {  
      "id": 1,  
      "title": "The Great Gatsby",  
      "author": "F. Scott Fitzgerald",  
      "isbn": "9780743273565",  
      "category": "Fiction",  
      "quantity": 5,  
      "available": 3,  
      "published_year": 1925,  
      "publisher": "Scribner",  
      "description": "A classic American novel...",  
      "tags": "classic, american, 1920s",  
      "location": "Shelf A-1"  
    }  
  ]  
}
```

2. Search Books

http

GET /includes/api_books.php?action=search&query=gatsby

Query Parameters:

- `query`: Search term
- `category`: Filter by category (optional)

Response:

json

```
{
  "success": true,
  "results": [
    {
      "id": 1,
      "title": "The Great Gatsby",
      "author": "F. Scott Fitzgerald",
      "available": 3,
      "category": "Fiction"
    }
  ]
}
```

3. Get Book by ID

http

GET /includes/api_books.php?action=get&id=1

Response:

json

```
{
  "success": true,
  "book": {
    "id": 1,
    "title": "The Great Gatsby",
    "author": "F. Scott Fitzgerald",
    "isbn": "9780743273565",
    "quantity": 5,
    "available": 3
  }
}
```

4. Add New Book (Admin Only)

http

POST /admin/add_book.php

Request Body:

```
json

{
  "title": "To Kill a Mockingbird",
  "author": "Harper Lee",
  "isbn": "9780061120084",
  "category": "Fiction",
  "quantity": 10,
  "available": 10,
  "published_year": 1960,
  "publisher": "J.B. Lippincott & Co.",
  "description": "A gripping tale of moral courage...",
  "tags": "classic, southern, justice",
  "location": "Shelf B-3"
}
```

Response:

json

```
{  
  "success": true,  
  "message": "Book added successfully",  
  "book_id": 45  
}
```

5. Update Book (Admin Only)

```
http  
  
PUT /admin/edit_book.php
```

Request Body:

```
json  
  
{  
  "id": 1,  
  "quantity": 7,  
  "available": 5,  
  "location": "Shelf A-2"  
}
```

6. Delete Book (Admin Only)

```
http  
  
DELETE /admin/books.php?action=delete&id=1
```

Response:

```
json  
  
{  
  "success": true,  
  "message": "Book deleted successfully"  
}
```

Patron Management Endpoints

1. Get All Patrons (Admin Only)

http

GET /admin/patron_directory.php?action=list

Response:

json

```
{
  "success": true,
  "patrons": [
    {
      "id": 1,
      "student_id": "2021-001",
      "name": "Jane Smith",
      "email": "jane.smith@email.com",
      "phone": "09171234567",
      "role": "client",
      "account_status": "active"
    }
  ]
}
```

2. Get Patron by ID

http

GET /admin/patron_profile.php?id=1

3. Add New Patron (Admin Only)

http

POST /admin/add_patron.php

Request Body:

json

```
{
  "student_id": "2024-001",
  "name": "Alice Johnson",
  "email": "alice.johnson@email.com",
  "phone": "09181234567",
  "address": "123 Main Street, Manila",
  "role": "client",
  "account_status": "active"
}
```

Response:

json

```
{
  "success": true,
  "message": "Patron added successfully",
  "patron_id": 25
}
```

4. Update Patron (Admin Only)

http

PUT /admin/edit_patron.php

Request Body:

json

```
{
  "id": 1,
  "phone": "09191234567",
  "address": "456 New Address, Quezon City",
  "account_status": "active"
}
```

5. Check Patron Borrowing Limit

http

GET /admin/check_patron_limit.php?patron_id=1

Response:

json

```
{
  "success": true,
  "can_borrow": true,
  "current_count": 2,
  "max_limit": 5,
  "has_overdue": false,
  "unpaid_fines": 0.00
}
```

Borrowing Endpoints

1. Create Borrow Request (Faculty)

http

POST /faculty/borrow_request.php

Request Body:

json

```
{
  "book_id": 5,
  "patron_id": 12,
  "borrow_days": 30,
  "notes": "Need for research project"
}
```

Response:

json

```
{
  "success": true,
  "message": "Borrow request submitted successfully",
  "borrowing_id": 87,
  "status": "pending"
}
```

2. Approve Borrow Request (Admin)

```
http
POST /admin/borrowing.php?action=approve
```

Request Body:

```
json
{
  "borrowing_id": 87
}
```

Response:

```
json
{
  "success": true,
  "message": "Borrow request approved",
  "due_date": "2025-01-30"
}
```

3. Reject Borrow Request (Admin)

```
http
POST /admin/borrowing.php?action=reject
```

Request Body:

json

```
{
  "borrowing_id": 87,
  "reason": "Book reserved for another patron"
}
```

4. Return Book

http

POST /admin/borrowing.php?action=return

Request Body:

json

```
{
  "borrowing_id": 87,
  "return_date": "2025-01-25"
}
```

Response:

json

```
{
  "success": true,
  "message": "Book returned successfully",
  "fine_amount": 0.00
}
```

5. Get Borrowing History

http

GET /faculty/my_borrowings.php?patron_id=12

Response:

json

```
{
  "success": true,
  "borrowings": [
    {
      "id": 87,
      "book_title": "Database Systems",
      "book_author": "R. Elmasri",
      "borrow_date": "2025-01-01",
      "due_date": "2025-01-30",
      "return_date": null,
      "status": "borrowed",
      "days_until_due": 5
    }
  ]
}
```

6. Get Pending Requests (Admin)

http

GET /admin/borrowing.php?action=pending

Response:

json

```
{
  "success": true,
  "pending_requests": [
    {
      "borrowing_id": 88,
      "book_title": "Advanced Algorithms",
      "patron_name": "Dr. Maria Santos",
      "request_date": "2025-01-20",
      "requested_days": 30
    }
  ]
}
```

Reservation Endpoints

1. Create Reservation

http

POST /faculty/reserve_book.php

Request Body:

json

```
{  
  "book_id": 10,  
  "user_id": 15,  
  "patron_id": 12,  
  "notes": "Needed for next semester"  
}
```

Response:

json

```
{  
  "success": true,  
  "message": "Book reserved successfully",  
  "reservation_id": 23,  
  "expiry_date": "2025-01-28"  
}
```

2. Get My Reservations

http

GET /faculty/reservations.php?user_id=15

Response:

json

```
{
  "success": true,
  "reservations": [
    {
      "id": 23,
      "book_title": "Machine Learning Basics",
      "reservation_date": "2025-01-25",
      "expiry_date": "2025-01-28",
      "status": "pending"
    }
  ]
}
```

3. Confirm Reservation (When Ready)

http

POST /faculty/confirm_reservation.php

Request Body:

json

```
{
  "reservation_id": 23
}
```

4. Cancel Reservation

http

DELETE /faculty/reservations.php?action=cancel&id=23

Fine Management Endpoints

1. Get All Fines (Admin)

http

GET /admin/fines.php?action=list

Response:

```
json
{
  "success": true,
  "fines": [
    {
      "id": 5,
      "patron_name": "John Doe",
      "book_title": "Physics Fundamentals",
      "amount": 50.00,
      "amount_paid": 0.00,
      "days_overdue": 10,
      "status": "unpaid",
      "created_at": "2025-01-15"
    }
  ]
}
```

2. Process Fine Payment

http

POST /admin/fines.php?action=pay

Request Body:

```
json
{
  "fine_id": 5,
  "amount_paid": 50.00
}
```

Response:

json

```
{  
  "success": true,  
  "message": "Payment recorded successfully",  
  "remaining_balance": 0.00  
}
```

3. Waive Fine (Admin)

```
http  
  
POST /admin/fines.php?action=waive
```

Request Body:

```
json  
  
{  
  "fine_id": 5,  
  "reason": "Student was hospitalized"  
}
```

Analytics Endpoints

1. Get Dashboard Statistics (Admin)

```
http  
  
GET /admin/analytics.php?action=dashboard
```

Response:

```
json
```

```
{
  "success": true,
  "stats": {
    "total_books": 1250,
    "total_patrons": 450,
    "active_borrowings": 87,
    "overdue_books": 12,
    "pending_requests": 5,
    "total_fines": 2500.00,
    "unpaid_fines": 750.00
  }
}
```

2. Get Borrowing Trends

http

GET /admin/analytics.php?action=trends&period=monthly

Response:

```
json
{
  "success": true,
  "trends": [
    {
      "month": "January",
      "total_borrowed": 145,
      "total_returned": 132,
      "overdue_count": 8
    }
  ]
}
```

3. Get Popular Books

http

```
GET /admin/analytics.php?action=popular&limit=10
```

Response:

```
json
{
  "success": true,
  "popular_books": [
    {
      "book_id": 25,
      "title": "Introduction to Algorithms",
      "author": "Cormen et al.",
      "borrow_count": 45
    }
  ]
}
```

Testing with Postman

Complete Postman Testing Guide

Setup Postman Environment

1. Open Postman
2. Click "Environments" (left sidebar)
3. Click "+" to create new environment
4. Name: "Library System - Local"
5. Add variables:
 - `baseUrl`: `http://localhost/Library`
 - `adminToken`: (will be set after login)
 - `facultyToken`: (will be set after login)

Test Collection Structure

Create a new collection: **"Library Management System API"**

Organize with folders:

- Authentication
 - Books
 - Patrons
 - Borrowing
 - Reservations
 - Fines
 - Analytics
-

Test 1: User Registration

Setup:

1. Method: `POST`
2. URL: `{{baseUrl}}/auth/register.php`
3. Headers:
 - `Content-Type`: `application/json`
4. Body (raw JSON):

```
json

{
  "username": "faculty_test",
  "password": "TestPass123!",
  "full_name": "Dr. Test Faculty",
  "email": "faculty.test@library.com",
  "role": "faculty"
}
```

Expected Response (201 Created):

```
json
```

```
{
  "success": true,
  "message": "Registration successful",
  "user_id": 25
}
```

Save Response Data: In Tests tab, add:

```
javascript

pm.test("Registration successful", function() {
  pm.response.to.have.status(201);
  var jsonData = pm.response.json();
  pm.expect(jsonData.success).to.eql(true);
  pm.environment.set("test_user_id", jsonData.user_id);
});
```

Test 2: Admin Login

Setup:

1. Method:
2. URL:
3. Body:

```
json

{
  "username": "admin",
  "password": "password"
}
```

Expected Response:

```
json
```

```
{
  "success": true,
  "message": "Login successful",
  "user": {
    "id": 1,
    "username": "admin",
    "role": "admin"
  }
}
```

Tests:

javascript

```
pm.test("Login successful", function() {
  pm.response.to.have.status(200);
  var jsonData = pm.response.json();
  pm.expect(jsonData.success).to.eql(true);
  pm.expect(jsonData.user.role).to.eql("admin");
});
```

Test 3: Add New Book

Setup:

1. Method:
2. URL:
3. Body:

json

```
{
  "title": "Clean Code",
  "author": "Robert C. Martin",
  "isbn": "9780132350884",
  "category": "Programming",
  "quantity": 5,
  "available": 5,
  "published_year": 2008,
  "publisher": "Prentice Hall",
  "description": "A handbook of agile software craftsmanship",
  "tags": "programming, clean code, best practices",
  "location": "Shelf C-5"
}
```

Expected Response:

```
json

{
  "success": true,
  "message": "Book added successfully",
  "book_id": 101
}
```

Tests:

```
javascript

pm.test("Book added", function() {
  var jsonData = pm.response.json();
  pm.expect(jsonData.success).to.eql(true);
  pm.environment.set("test_book_id", jsonData.book_id);
});
```

Test 4: Search Books

Setup:

1. Method: `GET`
2. URL: `{{baseUrl}}/includes/api_books.php?action=search&query=clean code`

Expected Response:

```
json

{
  "success": true,
  "results": [
    {
      "id": 101,
      "title": "Clean Code",
      "author": "Robert C. Martin",
      "available": 5
    }
  ]
}
```

Test 5: Add Patron

Setup:

1. Method: `POST`
2. URL: `{{baseUrl}}/admin/add_patron.php`
3. Body:

```
json

{
  "student_id": "2025-TEST-001",
  "name": "Test Student",
  "email": "test.student@library.com",
  "phone": "09171234567",
  "address": "123 Test Street, Manila",
  "role": "client",
  "account_status": "active"
}
```

Expected Response:

```
json
```

```
{
  "success": true,
  "message": "Patron added successfully",
  "patron_id": 50
}
```

Tests:

javascript

```
pm.test("Patron added", function() {
  var jsonData = pm.response.json();
  pm.environment.set("test_patron_id", jsonData.patron_id);
});
```

Test 6: Faculty Borrow Request

Setup:

1. Method: **POST**
2. URL: **{{baseUrl}}/faculty/borrow_request.php**
3. Body:

json

```
{
  "book_id": 101,
  "patron_id": 50,
  "borrow_days": 30,
  "notes": "Required for programming course"
}
```

Expected Response:

json

```
{
  "success": true,
  "message": "Borrow request submitted",
  "borrowing_id": 205,
  "status": "pending"
}
```

Tests:

```
javascript

pm.test("Request submitted", function() {
  var jsonData = pm.response.json();
  pm.expect(jsonData.status).to.eql("pending");
  pm.environment.set("test_borrowing_id", jsonData.borrowing_id);
});
```

Test 7: Approve Borrow Request (Admin)

Setup:

1. Method:
2. URL:
3. Body:

```
json

{
  "borrowing_id": 205
}
```

Expected Response:

```
json

{
  "success": true,
  "message": "Request approved",
  "due_date": "2025-02-25"
}
```

Test 8: Check Patron Borrowing Limit

Setup:

1. Method: `GET`
2. URL: `{{baseUrl}}/admin/check_patron_limit.php?patron_id=50`

Expected Response:

```
json
{
  "success": true,
  "can_borrow": true,
  "current_count": 1,
  "max_limit": 5,
  "has_overdue": false,
  "unpaid_fines": 0.00
}
```

Test 9: Create Book Reservation

Setup:

1. Method: `POST`
2. URL: `{{baseUrl}}/faculty/reserve_book.php`
3. Body:

```
json
{
  "book_id": 25,
  "user_id": 15,
  "patron_id": 12,
  "notes": "Will pick up next week"
}
```

Expected Response:

json

```
{
  "success": true,
  "message": "Book reserved",
  "reservation_id": 15,
  "expiry_date": "2025-02-03"
}
```

Test 10: Return Book

Setup:

1. Method:
2. URL:
3. Body:

json

```
{
  "borrowing_id": 205,
  "return_date": "2025-02-20"
}
```

Expected Response:

json

```
{
  "success": true,
  "message": "Book returned",
  "fine_amount": 0.00
}
```

Test 11: Get Dashboard Analytics

Setup:

1. Method:

2. URL: `{{baseUrl}}/admin/analytics.php?action=dashboard`

Expected Response:

```
json
{
  "success": true,
  "stats": {
    "total_books": 101,
    "total_patrons": 50,
    "active_borrowings": 25,
    "overdue_books": 3,
    "pending_requests": 2
  }
}
```

Test 12: Process Fine Payment

Setup:

1. Method: `POST`
2. URL: `{{baseUrl}}/admin/fines.php?action=pay`
3. Body:

```
json
{
  "fine_id": 5,
  "amount_paid": 25.00
}
```

Expected Response:

```
json
{
  "success": true,
  "message": "Payment recorded",
  "remaining_balance": 25.00
}
```

Running Test Collection

Automated Testing:

1. Click "Runner" in Postman
 2. Select "Library Management System API" collection
 3. Click "Run Library Management..."
 4. Tests will execute in sequence
 5. View results and response times
-

User Workflows

Admin Workflows

Daily Operations Workflow

Morning Routine:

1. Login to admin dashboard
2. Check pending faculty requests
3. Review overdue books list
4. Check system notifications

Processing Borrow Requests:

1. Navigate to "Borrowing Management"
2. Click "Pending Requests" tab
3. Review each request:
 - Check book availability
 - Verify patron status
 - Check borrowing limit
4. Click "Approve" or "Reject"
5. Add notes if rejecting

6. System sends notification to faculty

Processing Returns:

1. Patron brings book to desk
2. Open "Borrowing Management"
3. Search for borrowing record
4. Click "Return Book"
5. System calculates fine (if overdue)
6. Process fine payment if applicable
7. Update book availability

Managing Fines Workflow

1. Navigate to "Fines Management"
 2. Filter by status (unpaid/partial)
 3. Select fine to process
 4. Options:
 - **Pay:** Enter amount, click "Record Payment"
 - **Waive:** Enter reason, click "Waive Fine"
 - **Partial:** Enter partial amount
 5. Save and generate receipt
-

Faculty Workflows

Borrowing Books Workflow

Search and Request:

1. Login to faculty portal
2. Click "Search Books"
3. Enter search criteria
4. Browse results
5. Click on book title for details

6. If available:
 - Click "Borrow Now" (instant)
7. If unavailable:
 - Click "Reserve" for waiting list

Request with Approval:

1. Click "Request to Borrow"
2. Select borrowing period (up to 30 days)
3. Add notes (optional)
4. Submit request
5. Wait for admin approval
6. Receive email notification
7. Pick up book within 24 hours

Checking Borrowing Status:

1. Navigate to "My Borrowings"
2. View tabs:
 - **Active:** Currently borrowed
 - **Pending:** Awaiting approval
 - **History:** Past borrowings
3. Check due dates
4. Request extension if needed

Managing Reservations Workflow

1. Navigate to "Reservations"
2. View reservation status:
 - **Pending:** Book not available
 - **Ready:** Book available for pickup
 - **Expired:** Reservation expired
3. When status is "Ready":

- Click "Confirm Reservation"
- Visit library within 3 days
- Complete checkout

4. To cancel:

- Click "Cancel Reservation"
 - Confirm cancellation
-

Client/Student Workflows

Searching and Borrowing

Simple Borrow:

1. Login to client portal
2. Click "Search Books"
3. Enter search term
4. Click "Borrow" on available book
5. System checks:
 - Borrowing limit (max 5 books)
 - No overdue books
 - No unpaid fines
6. If passed, book checked out immediately
7. Due date: 14 days from today

Checking Account:

1. Navigate to "Dashboard"
2. View widgets:
 - Currently Borrowed (X/5)
 - Due Soon (next 3 days)
 - Overdue Books (red alert)
 - Fine Balance
3. Click on any widget for details

System Configuration

Adjusting System Settings (Admin)

1. Navigate to "Settings"
2. Select category:
 - **Borrowing:** Days, limits
 - **Fines:** Rates, grace period
 - **Reservations:** Expiry days
 - **Security:** Lockout, attempts
 - **Features:** Enable/disable modules

Example Changes:

Default Borrow Days (Students): 14 → 21
Faculty Max Books: 10 → 15
Fine Per Day: 5.00 → 7.50
Max Login Attempts: 5 → 3

Troubleshooting

Common Issues & Solutions

Issue: "Cannot borrow - limit reached"

- **Cause:** Patron has maximum books borrowed
- **Solution:** Return books or wait for admin to increase limit

Issue: "Pending request not appearing"

- **Cause:** Database trigger failed
- **Solution:** Check MySQL triggers are enabled

Issue: "Fine not calculated"

- **Cause:** `auto_calculate_fines` setting disabled
- **Solution:** Enable in System Settings

Issue: "Book availability not updating"

- **Cause:** Trigger issue or manual override
- **Solution:** Run `/admin/fix_availability.php`

Issue: "Session expired too quickly"

- **Cause:** Low `session_timeout` value
- **Solution:** Increase in System Settings (default: 30 min)

Issue: "Cannot approve faculty request"

- **Cause:** Insufficient admin permissions
 - **Solution:** Verify admin role in users table
-

Security Best Practices

1. **Change Default Credentials:** Never use default admin password
 2. **Enable Two-Factor Authentication:** For admin accounts
 3. **Regular Backups:** Daily database backups recommended
 4. **Update Dependencies:** Keep PHP and MySQL updated
 5. **Monitor Login Attempts:** Review `login_history` regularly
 6. **Audit Activity Logs:** Check for suspicious activity
 7. **Use HTTPS:** In production environments
 8. **Limit Admin Accounts:** Only create when necessary
 9. **Strong Password Policy:** Enforce minimum requirements
 10. **Regular Password Changes:** Every 90 days
-

Maintenance Tasks

Daily

- Check pending requests
- Review overdue books
- Process fine payments

Weekly

- Database backup
- Review activity logs
- Clean expired sessions
- Update book availability

Monthly

- Generate reports
 - Analyze borrowing trends
 - Review system settings
 - Archive old data
 - Update book catalog
-

Support & Contact

For technical support:

- Check this documentation first
- Review error logs in MySQL
- Contact system administrator
- Submit bug reports with:
 - Error message
 - Steps to reproduce
 - Expected vs actual behavior

- Screenshots

Version: 2.1

Last Updated: December 31, 2024

Database Schema: Version 2.1 (UTF8MB4)

Maintained by: Development Team

Appendix

SQL Quick Reference

Check Pending Requests:

```
sql

SELECT * FROM view_pending_faculty_requests;
```

Find Overdue Books:

```
sql

SELECT * FROM view_overdue_books;
```

Check Patron Borrowing Stats:

```
sql

SELECT * FROM user_borrowing_stats WHERE user_id = 15;
```

Calculate Total Fines:

```
sql

SELECT SUM(amount - amount_paid) as total_unpaid
FROM fines
WHERE status IN ('unpaid', 'partial');
```

Get Book Availability:

```
sql
```

```
SELECT * FROM view_book_availability;
```

File Permissions

Ensure proper permissions (Linux/Mac):

```
bash

chmod 755 Library/
chmod 644 Library/*.php
chmod 755 Library/admin/
chmod 755 Library/auth/
chmod 755 Library/client/
chmod 755 Library/faculty/
chmod 755 Library/includes/
```

PHP Configuration

Recommended `php.ini` settings:

```
ini

upload_max_filesize = 10M
post_max_size = 10M
max_execution_time = 300
memory_limit = 256M
session.gc_maxlifetime = 1800
```

End of Documentation